In this assessment, students will learn about creating small REST projects. The goal is to understand general concepts and behaviors related to REST API.

1. Installing Flask Dependencies

Throughout the exercises we will use python and flask to create a REST API. To install flask on ubuntu just run the command:

```
$pip3 install flask
```

2. First REST Interface

To create a "Hello World" Rest API in Flask just run the following code:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'
```

This will create an interface in http://localhost:5000/ that returns "Hello World" for a HTTP GET.

To invoke the python you can run on older versions:

```
$python3  app.py
```

Or in newer python versions:

```
$ export FLASK_APP=app.py
$python3 -m flask run
```

3. REST Interface routing

We may want to redirect our traffic to different business logic. The following example describe that scenario:

```
@app.route('/')
def index():
    return 'Index Page'

@app.route('/hello')
def hello():
    return 'Hello, World'
```

4. URL variables

You can have variables on the URL useful to run logic for distinct resource identifiers:

```
from markupsafe import escape

@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return 'User %s' % escape(username)

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # show the post with the given id, the id is an integer
    return 'Post %d' % post_id

@app.route('/path/<path:subpath>')
def show_subpath(subpath):
    # show the subpath after /path/
    return 'Subpath %s' % escape(subpath)
```

5. Other HTTP Methods

To achieve a CRUD API we need other HTTP verbs beside the GET. The following code describes how to use other HTTP Verbs:

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

6. Creating a more complex REST API

Try to create a REST API to manage a a group of stores. Each store might have distinct products and the group might have multiple stores. Additionally, each product might have a distinct price on each store.

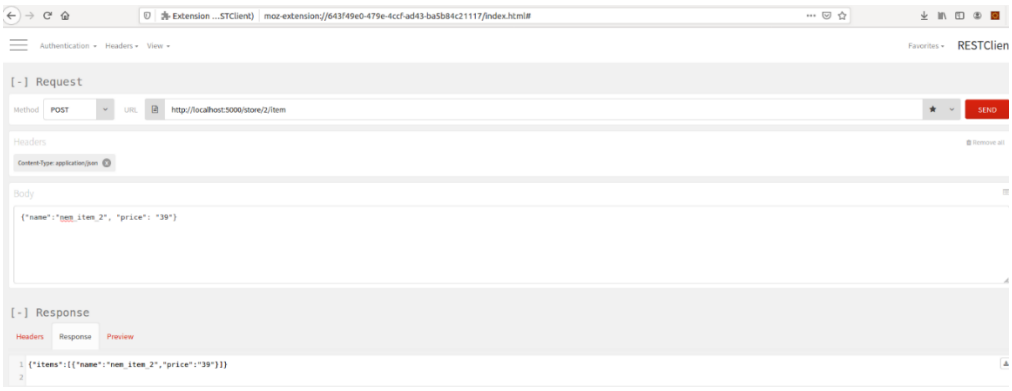
Try to implement the following endpoints:

Method HTTP	Endpoint	Description
GET	/	Get html webpage
POST	/store	Create a new store. Receives a json with the format: {"name":"name_of_store"}
GET	/store/<name>	Get Store info
GET	/store	Get All store info
POST	/store/<name>/item	Create a new item on the store <name>: Accepts json: {"name":"nem_item_1", "price": "39"}
GET	/store/<name>/item	Get all items from store <name>

Other methods can be implemented – e.g. DELETE store

Note: The exercise resolution is available here

To test the API you can use POSTMAN or some browser extension like REST client for firefox:



Try also the following items:

- connect to a Database using SQLAlchemy
- Swagger to create Documentation
- Flask-Restful to create classes and more isolated methods/classes